

Exam Operating Systems - January 2026

Question 1: Caching (2 pts)

The witch's gingerbread house in Hansel and Gretel doesn't only contain candy but also has a sophisticated child-tracking system equipped with a local cache. The operations performed during a normal searching cycle are given in the code below.

```
1. int a[1024], b[256];
2. for(i=0; i < 256; i++){
3.     s = 0;
4.     for(j = 0; j < 2; j++){
5.         s = s + a[i+j];
6.     }
7.     b[i] = s;
8. }
```

The cache uses 8B blocks and 4B memory words. An integer is 4B long. The arrays a and b are stored sequentially in memory (no gaps). You may assume that the variables i , j and s are stored in registers. This cache is 1kB direct-mapped and has an access time of 20 ns. A cache-miss takes 80ns.

- Calculate the hit rate. (1 pt)
- Provide the mean access time of this system. (1 pt)

Question 2: Scheduling (5 pts)

Getting Cinderella to the ball was no easy feat! The Fairy Godmother was fiercely working towards this goal. She had a clear view of what had to be done; all tasks were known beforehand. Due to the magic's expiration at midnight, she had strict deadlines.

ID	Task	Duration	Deadline
A	Mend the Dress	5 hours	14:00
B	Transform Mice to Footmen	3 hours	19:00
C	Conjure the Glass Slippers	2 hours	18:00
D	Find a Suitable Pumpkin	9 hours	8:00
E	Coach Cinderella on Etiquette	6 hours	24:00 (midnight)

start time = 00:00
so in total: 24 hours to
schedule everything

- Provide the best schedule for $1 || L_{\max}$ and discuss the used algorithm. (1 pt)
- Now optimize $1 || \sum U_j$ and discuss the algorithm. (1 pt)
- How could we improve both above algorithms when allowing preemption? (1 pt)

To hone their party skills, the the Fairy and the Enchantress from the neighboring kingdom both prepare an identical ball each year. Both balls require the same n tasks. We denote p_i^F and p_i^E as the (possibly different) preparation times for each task i by the Fairy and the Enchantress respectively. Furthermore, they both have (possibly different) deadlines. Fortunately, they can collaborate on a task, resulting in a time of $\frac{p_i^F + p_i^E}{2}$ for both (this doesn't change either deadline). For instance, "cleaning the chandelier" takes 5 hours for the Fairy and 9 hours for the Enchantress; working together results in 7 hours of work for both. Collaboration can be done asynchronously.

- Describe an algorithm that minimizes $L_{\max}$ (which is a sum of delays of both parties) and motivate your solution. Be transparent and complete in your explanation of its weaknesses and strengths, points are given based on the accuracy and efficiency of the algorithm. (2 pts)

Question 3: Disk management (2 pts)

Rapunzel's tower is so tall that it is organized by floors where various princes wait to be granted access. The tower uses an elevator system to efficiently (and fairly) serve all waiting suitors. The current location of the elevator is floor 1200 and its last location was floor 1000. The tower uses the C-LOOK algorithm to manage prince transportation.

- a) In which order will the elevator handle the following floor requests: 1100, 1300, 700, 1700? (1 pt)
- b) Compare this technique with SCAN and FSCAN, what are the pros and cons? (1 pt)

Question 4: Disk management (5 pts)

When the good fairies were preparing to protect Sleeping Beauty, they faced a range of difficulties. The enchantments guarding the princess generate lots of data over the 100-year slumber. Unfortunately, the available storage disks are fragile and prone to magical exhaustion, which leads to loss of valuable protection records. To counter this, the fairies agreed to use RAID. Consider a 6-disk setup with the following specifications:

Capacity	Mean Seek Latency	Mean Rotational Delay	Transfer Speed
120 GB	7ms	3ms	60MB/sec

- a) Concisely discuss all RAID levels and outline their suitability for this task. Importantly, fairies may fall into slumber themselves if they channel too many writes. (0.5 pt)
- b) Using RAID 5, how long does storing a single byte take on an arbitrary location? (1 pt)
- c) The century has ended. Using RAID 5, how long does it take to read all information stored sequentially? (1 pt)
- d) A new generation of disks have a capacity of 1TB. 6 of these new disks are added to the previous setup using a RAID 5 + 1 setup, what will be the new effective capacity? (1 pt)
- e) Compare the robustness (in detail) of this new system to the old system. (0.5 pt)
- f) Discuss any alternatives to this RAID setup that could be better suited. (1 pt)

Question 5: Concurrency (6 pts)

The seven dwarfs had a difficult life while mining their riches. Their endeavour consisted of two main activities: mining gems and sharing the workshop.

Their first challenge is accessing the mine shaft, which can only accommodate dwarfs moving in one direction at a time. Since dwarfs need to exit sometimes, this narrow passage is bottlenecked.

Consider the following implementations to resolve this issue. Each dwarf is represented as a thread that signals which direction they are moving in with the procedures *enter(direction)* and *leave(direction)*. Only between these two calls may the dwarf use the shaft in the given direction.

In this implementation, the direction is a number equal to 0 or 1. *VarLock* and *shaftLock* are binary semaphores that are initialized on 1 and *dwarfs* is an array consisting of two integers. All variables are shared globally.

1. <i>enter</i> (direction) {	1. <i>leave</i> (direction){
2. <i>wait</i> (varLock);	2. <i>wait</i> (varLock);
3. dwarfs[direction] += 1;	3. dwarfs[direction] -= 1;
4. if(dwarfs[direction] == 1){	4. if(dwarfs[direction] == 0){
5. <i>wait</i> (shaftLock);	5. <i>signal</i> (shaftLock);
6. }	6. }
7. <i>signal</i> (varLock);	7. <i>signal</i> (varLock);
8. }	8. }

The only difference in this implementation is its use of 2 binary semaphores for *VarLock*. Both are initialized at one.

1. <i>enter</i> (direction) {	1. <i>leave</i> (direction){
2. <i>wait</i> (varLock[direction]);	2. <i>wait</i> (varLock[direction]);
3. dwarfs[direction] += 1;	3. dwarfs[direction] -= 1;
4. if(dwarfs[direction] == 1){	4. if(dwarfs[direction] == 0){
5. <i>wait</i> (shaftLock);	5. <i>signal</i> (shaftLock);
6. }	6. }
7. <i>signal</i> (varLock[direction]);	7. <i>signal</i> (varLock[direction]);
8. }	8. }

a) For both given implementations of *enter* and *leave*, provide an intuitive explanation of the algorithm. Likewise, discuss what could go wrong and when that happens. (2 pts)

Back at the workshop, the other dwarfs polish the gems. While multiple dwarfs can work on polishing simultaneously, they must clear out when Snow White needs to clean.

b) Assume only Snow White performs the cleaning. Provide an implementation of *polisher* and *cleaner* threads. Ensure no deadlocks or starvation. Explain and motivate your answer. (2 pts)

c) Change the previous implementation such that multiple helpers can assist Snow White with cleaning (using the same *cleaner* implementation). Avoid starvation and deadlocks, explain. (2 pts)